fonction de hachage

Karim Madani

🎓 1. Définition simple (niveau débutant)

👉 Une **fonction de hachage** transforme n'importe quelle donnée en une **empreinte unique** (appelée *hash*).

🧠 Analogie simple :

Imagine une machine :

- Tu mets un message dedans 📩
- Elle te donne une “empreinte digitale” 🧬

👉 Même si le message est long → le résultat est court et fixe

⚙️ 2. Explication technique (niveau avancé)

Une fonction de hachage est une fonction mathématique :

$h:\{0,1\}^* \rightarrow \{0,1\}^n$ h:{0,1}∗→{0,1}n

👉 Elle prend :

- une entrée de taille variable
- et produit une sortie de taille fixe

🔐 Propriétés importantes

1. **Déterministe**
    a. Même entrée → même hash
2. **Résistance à la préimage**
    a. Impossible de retrouver le message original à partir du hash
3. **Résistance aux collisions**
    a. Très difficile de trouver deux messages différents avec le même hash
4. **Effet avalanche**
    a. Un petit changement → hash totalement différent

🧪 Exemples connus

- MD5 (cassé ❌)
- SHA-1 (faible ❌)
- SHA-256 (sécurisé ✅)

🔥 3. Exemple concret

Message :

`bonjour`

Hash (ex: SHA-256) :

`9f86d081884c7d659a2feaa0c55ad015...`

👉 Si tu changes juste :

`Bonjour`

➡️ Le hash devient complètement différent !

🛠️ 4. Applications en cybersécurité

🔑 1. Stockage des mots de passe

- On ne stocke pas le mot de passe
- On stocke son hash

👉 Vérification :

- utilisateur entre mot de passe
- système calcule hash
- compare avec base

📦 2. Intégrité des données

- Vérifier si un fichier a été modifié

🔏 3. Signatures numériques

- Garantir authenticité + intégrité

⛓️ 4. Blockchain

- Chaque bloc contient le hash du précédent

⚠️ 5. Limites / attaques

🚨 Attaques possibles :

- **Collision attack**
- **Rainbow tables**
- **Brute force**

👉 Solution :

- Utiliser du **salt**
- Utiliser des fonctions lentes (bcrypt, argon2)

📌 6. Résumé rapide

👉 Une fonction de hachage :

- Transforme une donnée → empreinte fixe
- Impossible à inverser
- Utilisée pour sécuriser mots de passe et données